

Report for Assignment 1: Code Generation

Wang Canhao 3036411009

1 Introduction

Large Language Models (LLMs) have shown great potential in solving complex problems, such as coding problems. Recent research has explored ways to increase LLMs' performance in complex problems, and multiple methods have proven useful for this task. For this project, the objective is to experiment with the code generation ability of the given LLM, Llama-3.1-8B-Instruct, and improve its performance with prompt engineering.

This project implements a basic method with [the HumanEval dataset](#) as the baseline. It applies the methods from two papers [1, 2] separately and then together to enhance the performance. The influence of hyperparameters (temperature and top_p) and evaluation metrics (pass@1, pass@3) are experimented with and recorded to analyze the effects of different methods compared with the baseline. Though the two methods show little improvement independently, the combined implementation achieves an over 10% absolute increase in pass@1 metric in comparison with the baseline.

The following report contains four sections. The first section describes useful ideas and conclusions from the papers. Then the actual implementations are provided. The third section is experiment and analysis, and the final one draws a conclusion and discusses the limitations.

2 Backgrounds

The two methods are Self-Evolve [2] and CodeT [1]. This section offers a brief overview of the major metric, ideas and notable conclusions.

2.1 Major metric

Pass@1 (pass at 1) is the major metric for evaluating and comparing the performance of different methods. It reflects the probability of a method solving the problem with just one attempt. For a clearer understanding, if k solutions are generated, pass@ k is the possibility that any of the k solutions solves the problem.

The evaluation code offers an unbiased method to estimate pass@ k :

$$Pass@k = E \left[1 - \frac{\binom{n-c}{k}}{\binom{n}{k}} \right]$$

Where n and c denote the numbers of total and correct solutions.

2.2 Reasoning and feedbacks

Teaching LLM to reason before generating solutions can lead to a more accurate and comprehensive understanding of the problems, supporting subsequent responses. Based on the knowledge, the model can derive solutions with higher quality and accuracy.

Moreover, allowing LLMs to refine their answers can further enhance the effectiveness of their solutions. Typically, human programmers may be unaware of minor issues such as boundary issues during their initial implementations, and they also need to repeat debugging and optimizing to pass all the tests. This applies to LLMs as well. With the feedback generated by evaluating the initial code with unit tests, LLMs can catch and fix their mistakes and produce better solutions.

Self-Evolve provides a prompt template for reasoning and then solving coding problems. It also conducts experiments to show that a single round of self-refinement can achieve substantial enhancements in accuracy for the HumanEval dataset, while additional iterations yield diminishing returns [2].

This project utilizes the prompt template and develops one round of self-refinement. However, the original dataset contains few test cases, and they should be tests for final evaluation rather than tests for intermediate refinement.

2.3 Larger test cases

To better evaluate the performance of solutions, CodeT [1] develops a method requiring LLM to generate more test cases. It then forms consensus sets and calculates the score of each set to find the set with the highest score, where better solutions are located. Given that not all generated test cases are correct, however, some toxic cases may elevate the score of consensus sets with poor solutions, leading to incorrect judgements.

3 Implementation

This section includes the major points in the implementations using different methods. Despite the evaluation codes, most of the code generation scripts are developed from scratch based on knowledge and ideas from the two papers [1, 2]

3.1 Zeroshot baseline

This method evaluates the basic performance of llama-3.1-8B-Instruct. The system prompt tells the model to solve the coding problems with Python, and the next user prompt provides the raw question without any additional instruction. Different pairs of temperature and top_p are tested to examine their relations with pass@k. Given that the estimated metric is more accurate with more samples, this paper chooses n = 5 for calculating pass@1 and pass@3. In addition, the evaluation code is slightly modified.

3.2 Self-evolve method

After experimenting with hyperparameters in the baseline, temperature and top_p are set as 0.8 for subsequent methods. For self-evolve, this paper refers to its prompt template and develops a three-step process. First, the program enquires the LLM about how to solve the problem. The LLM is required to output knowledge in natural language only. With the received knowledge, the program asks the model to generate a code solution without any explanation or natural language. The previous chat history (sub_prompt_1 and the 1stresponse) becomes necessary context and thus is concatenated with the 2ndsub_prompt, so the LLM can take the generated knowledge into account.

The solution from the 2nd response is examined with the first test case in each problem to generate test feedback. If the feedback shows the solution passes the test, the 2nd sub_prompt and its response (code solution) become the final prompt and output. If not, the program creates another sub_prompt with the whole code (the completed function and the test function with the test case) and refinement paddings and asks the LLM one more time for refinement (enough already [2]). According to the test results (wrong answer or syntax error), different padding texts are added behind the whole code to help the model understand its mistake better. The 3rd sub_prompt and its response become the final prompt and output.

3.3 CodeT method

For this method, the program first concatenates the canonical_solution in the dataset with the problem to construct a completion and then adds the test function (check(candidate)) and a line with a single “assert” to remind the LLM to generate more test cases in assertion format.

Different from the method in the paper which allows toxic test cases in the set [1], this project leverages the canonical_solution to check the generated tests and clear all toxic cases that the ground truth function cannot pass. Before saving the test set, manual cleaning and size reduction are applied to further its quality. Given no toxic cases exist, the simplified scoring method only needs to count how many cases each solution passes and select solutions with the top k highest pass count as outputs.

Moreover, inspired by the idea that the finetuned test set only needs to be generated once, and all the solutions can be examined by it, this project can divide the whole process into separate steps and create checkpoints for interim results. Then the program can continue from the latest checkpoint if unexpected errors or time and resource limits interrupt it.

Given that this method requires score ranking to select outperformed solutions, the program only selects 3 of the 5 generated solutions as its outputs.

3.4 Combined method

The final method combines the two above methods, self-evolve provides a more complete generation flow and codeT offers a larger and better test set to enhance the examination part of the former method. The combined method first generates solutions using the knowledge-then-code query process and then checks the correctness of each solution with the test set. Top k solutions are selected, and their respective wrong cases (if they exist) are recorded. After the examination, selected solutions need to go through self-refinement if they have wrong test cases. All the wrong cases can be sent together once because each of them is just an assertion line. Finally, solutions passing all tests or being refined become the output of the program.

Checkpoints are set right behind the time-consuming steps (enquiring the LLM and conducting the examination) in case of unexpected failure.

4 Experiment and analysis

This section shows the results of the experiments and discusses the factors which may contribute to the difference between different methods.

4.1 Influence of hyperparameters

In the “completions” module from OpenAI.chat, temperature and top_p are editable hyperparameters controlling the response of LLM.

Temperature denotes the randomness of the generated response: the lower the temperature, the more conservative LLM tends to be with the output. The following chart (Fig 1) illustrates the relation between accuracy metrics and temperature (data documented from baseline outputs)

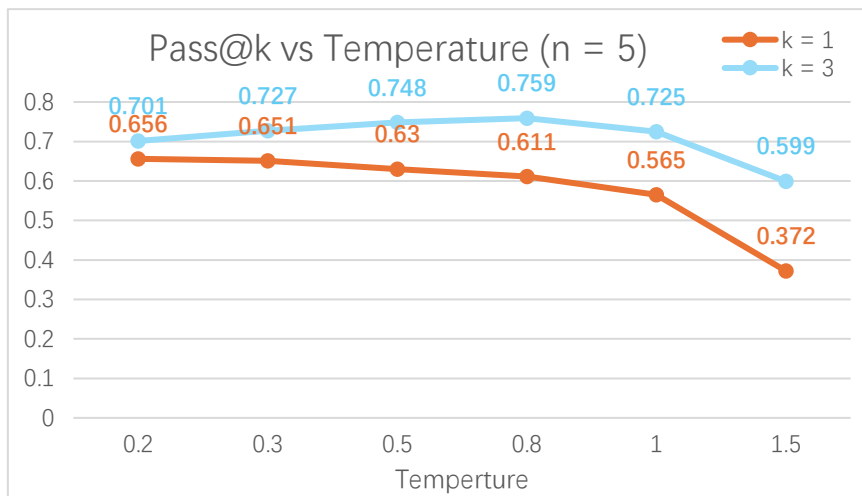


Figure 1: Pass@k derived from models with different temperature (5 total samples) [self-drawn]
The chart implies that, when temperature decreases, the accuracy increases and then

decreases after a threshold. For different pass@k metrics, a smaller k metric is more important because users generally want LLMs to solve problems with fewer attempts. However, a slightly larger k can indicate a more accurate trend because of larger C_n^k , which means more possible combinations can be selected from the same set. Therefore, this project chooses 0.8 for the final value of temperature.

Then another hyperparameter, top_p, is experimented. It denotes the proportion of options that a LLM considers when responding. The LLM sorts the probabilities of the outputs in descending order and calculates the accumulated probability. Once the value exceeds the threshold (top_p), the remaining part is discarded. Therefore, the lower the top_p, the fewer choices LLMs have and thus more stable output. Table 1 shows the experimented top_p values and their respective pass@k.

Table 1: Pass@1 and Pass@3 results with different top p values (temperature = 0.8)

Top p	Pass@1	Pass@3
1.0	0.620	0.746
0.8	0.654	0.746
0.6	0.648	0.694

The accuracy metrics also have a rise and then fall when top_p decreases. Therefore, 0.8 is set as the value of top_p for subsequent implementations.

4.2 Self-evolve

For the original evaluation script, solutions are combined with the problem before being concatenated with the test function. In other words, the beginning part (imported modules and the function definition) of the generated solution should be discarded because they are included in the original problem. However, both the baseline and self-evolve methods show a decline in accuracy. Therefore, this project modifies the evaluation process, removing the problem part and using the complete solution as completion. The changes in performance are recorded in Table 2.

Table 2: Pass@k of two methods with different evaluation methodology (t = 0.8, top p = 0.8)

Method	Original methodology	
	Pass@1	Pass@3
Baseline	0.572	0.693
Self-evolve	0.608	0.787
Method	Modified methodology	
	Pass@1	Pass@3
Baseline	0.654	0.746
Self-evolve	0.655	0.808

As shown in Table 2, modified methodology performs better. The reason may be that, the generated solutions sometimes import new modules like math and utilize them in

the function, and using the problem as the header of the completion cannot import these modules correctly, which leads to errors. After switching to the new evaluation script, self-evolve only achieves a mere improvement in pass@1.

4.3 CodeT

The results of codeT compared with former methods are shown in Table 3.

Table 3: Performance of different methods. (5 to 3) denotes that codeT generates 5 solutions and pick the best 3 of them as final outputs. (t = 0.8, top p = 0.8)

Method	Pass@1	Pass@3
Baseline	0.654	0.746
Self-evolve	0.655	0.808
CodeT (5 to 3)	0.634	0.683

The result of codeT is worse than the baseline. A possible reason is that the method only generates few samples for each problem, which hinders its filtering ability.

4.4 Combined method

Finally, the two implemented methods are combined. Table 4 shows the baseline and two experiments generating different quantities of samples.

Table 4: Performance of the combined method (t = 0.8, top p = 0.8), the result of (10 to 5) is more reliable because of the larger sample set.

Method	Pass@1	Pass@3
Baseline	0.654	0.746
Combined (5 to 3)	0.748 (+0.094)	0.848 (+0.102)
Combined (10 to 5)	0.765 (+0.111)	0.826 (+0.080)

An increase of nearly 11% absolute accuracy is achieved comparing with the baseline, showing the superiority of the combined result. It confirms that the test set generated by codeT can reinforce the testing part of self-evolve and thus produce better feedbacks for self-refinement.

5 Conclusions and limitations

This project refers to two methods from previous research to enhance the performance of the LLM when dealing with Python coding problems. Although, after this project utilizes their ideas and implement the methods respectively, little improvement in accuracy is achieved on llama-3.1-8B, the combined method yields promising results, suggesting its value on other LLMs.

Currently, the test set of codeT is manually cleaned to improve its quality. When

implementing this method to other datasets, using generated test set without manual modification may reduce its improvement, which needs to be further evaluated. Moreover, the developed method keeps the hyperparameters the same when requesting the LLM. More improvement may be achieved if the hyperparameters in each requesting step is modified respectively. There is still significant room for improvement in future work.

Reference

- [1] B. Chen *et al.*, "CodeT: Code Generation with Generated Tests," ed. Ithaca: Cornell University Library, arXiv.org, 2022.
- [2] S. Jiang, Y. Wang, and Y. Wang, "SelfEvolve: A Code Evolution Framework via Large Language Models," 2023, doi: 10.48550/arxiv.2306.02907.